

A Project Report On Online Food Ordering System

DEVELOPED BY:

IT088 – YASHIL PANDYA

IT076 – HARSH MODI

IT085 – VANSI PANCHAL

Guided By
Internal Guide:
Prof. Shweta Jambukia

Department of Information Technology Faculty
of Technology
DD University



Department of Information Technology Faculty of Technology,
Dharm Singh Desai University College Road, Nadiad-387001
March-2026

CERTIFICATE

This is to certify that the project entitled “Online Food Ordering System” is a bonafide report of the work carried out by

- 1) Yashil Pandya Student ID No: 24ITUOZ088
- 2) Harsh Modi Student ID No: 24ITUBS076
- 3) Vansh Panchal Student ID No: 24ITUBS085

of Department of Information Technology, semester IV, under the guidance and supervision for the subject Database Management System. They were involved in Project training during the academic year 2025-2026.

Prof. Shweta Jambukia

Project Guide, Department of Information Technology,
Faculty of Technology,
Dharmsinh Desai University, Nadiad

Date:08/03/2026

Prof. Vipul Dabhi

Head, Department of Information Technology

INDEX

I.Certificate.....	I
II. Commendation	II
1.SYSTEM OVERVIEW	4
a. Current system.....	4
b. Advantages of the Proposed system (over current).....	4
2.E-R Diagram.....	5
3.Relation Schema.....	6
4.Data DICTIONARY.....	7
5.DATABASE IMPLEMENTATION.....	12
5.1 Create Schema	12
5.2 Insert Data values.....	15
5.3 Queries.....	20
5.4 Queries (Based on Joins & Sub-Queries).....	20
5.5 PL/SQL Blocks (Views)	25
5.6 Functions & Triggers.....	26
5.7 Cursors.....	30

1.SYSTEM OVERVIEW

Data is an important part of the computer industry. Data refers to a collection of facts gathered from observations, transactions, or processes within a computer system. It may include numbers, text, images, or other information. Data is considered the lowest level of abstraction, from which useful information and knowledge are derived.

An Online Food Ordering System (OFOS) is a web-based or mobile-based system that allows customers to order food from restaurants through the internet. This system helps manage information related to customers, restaurants, menus, orders, and payments.

Using this system, customers can browse the menu, select food items, place orders, and make payments online. Once an order is placed, the restaurant receives the order details and starts preparing the food. The system also allows users to track the status of their orders until delivery.

The system can generate several types of reports, such as:

- Daily or monthly order reports
- Sales reports
- Customer order history
- Popular food items report

The system also provides notifications to customers about order confirmation, preparation, and delivery status. Therefore, the Online Food Ordering System makes the process of ordering food faster, easier, and more efficient for both customers and restaurants.

1.2 ADVANTAGES OF THE PROPOSED SYSTEM

1.Provide Easy Food Ordering – The main objective of the system is to allow customers to easily order food online from restaurants.

2.Reduce Manual Work – The system reduces manual order taking and paperwork in restaurants.

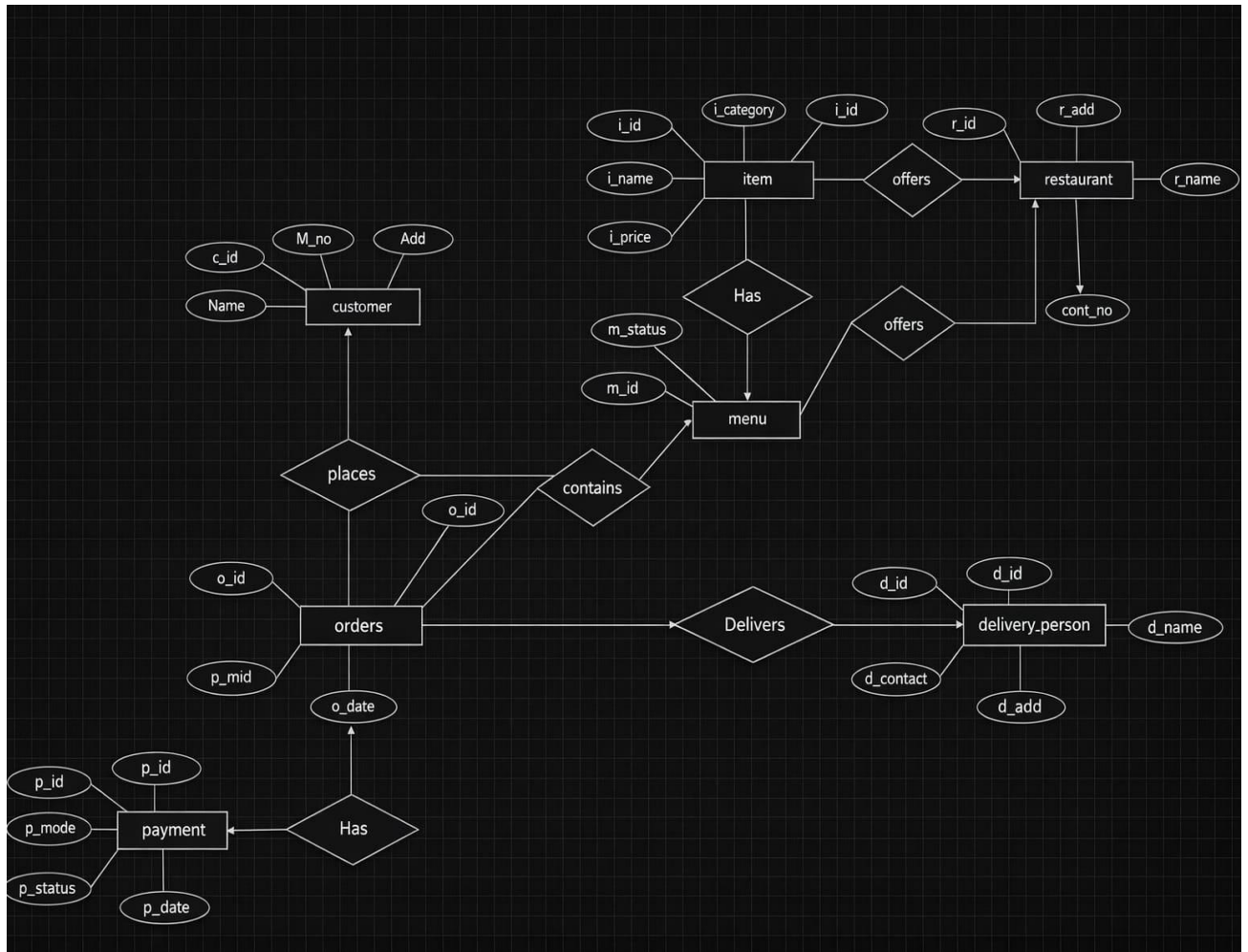
3.Improve Customer Experience – It provides a simple interface where customers can quickly browse menus and place orders.

4.Efficient Order Management – The system helps restaurants manage orders, payments, and customer information efficiently.

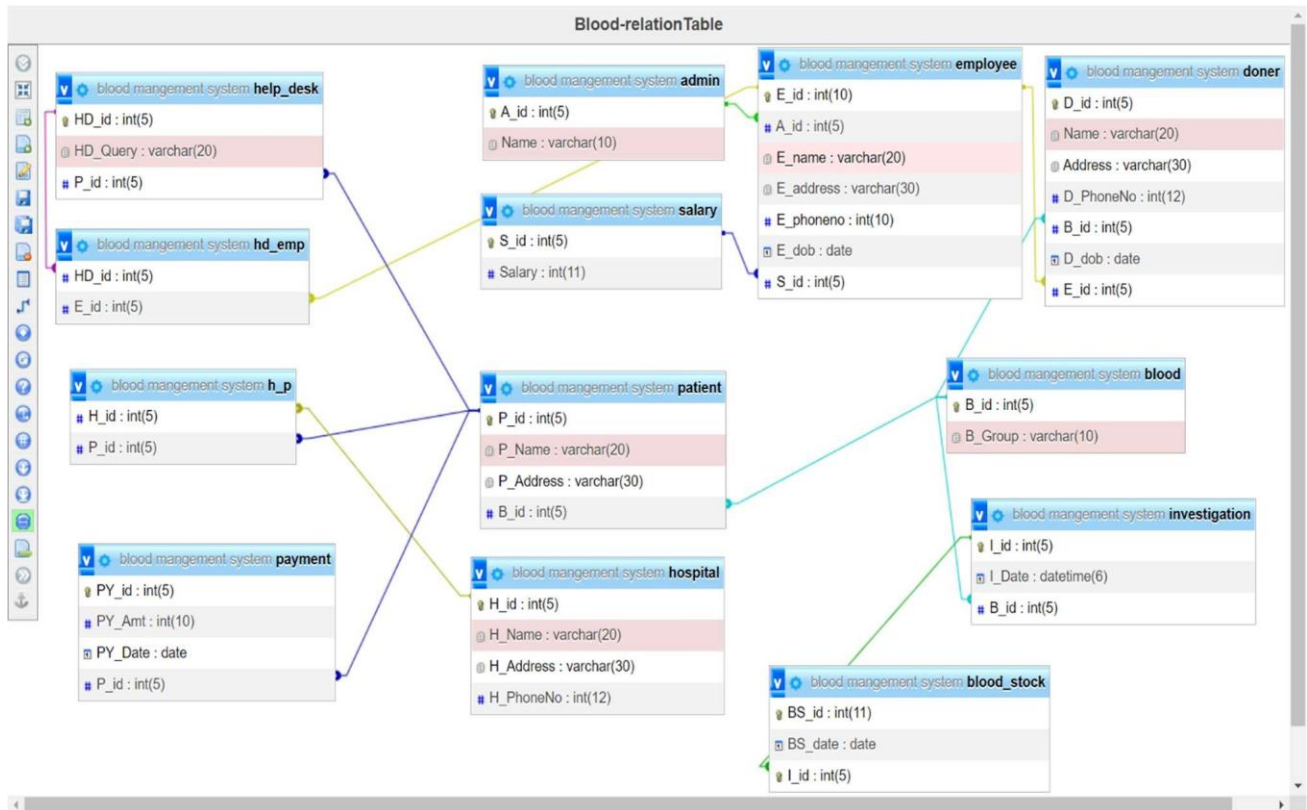
5.Provide Order Tracking – Customers can track the status of their orders from preparation to delivery.

6.Generate Reports – The system can generate reports such as daily sales, order history, and popular food items.

2. ENTITY-RELATIONSHIP MODEL



3. RELATIONAL SCHEMA



4. Data Dictionary

4.1 Customer

```
postgres=# \d customer;
Table "public.customer"
Column |          Type          | Collation | Nullable | Default
-----+-----+-----+-----+-----
c_id   | character varying(5)   |           | not null |
c_name | character varying(20)  |           | not null |
m_no   | numeric(10,0)          |           | not null |
email  | character varying(20)  |           |          |
c_add  | character varying(20)  |           |          |
Indexes:
    "customer_pkey" PRIMARY KEY, btree (c_id)
Referenced by:
    TABLE "orders" CONSTRAINT "orders_c_id_fkey" FOREIGN KEY (c_id) REFERENCES customer(c_id)
    TABLE "orderss" CONSTRAINT "orderss_c_id_fkey" FOREIGN KEY (c_id) REFERENCES customer(c_id)
```

4.2 Item

```
postgres=# \d item;
Table "public.item"
Column |          Type          | Collation | Nullable | Default
-----+-----+-----+-----+-----
i_id   | character varying(5)   |           | not null |
i_name | character varying(20)  |           | not null |
i_price | numeric(5,0)           |           |          |
i_category | character varying(5) |           |          |
r_id   | character varying(5)   |           |          |
Indexes:
    "item_pkey" PRIMARY KEY, btree (i_id)
Foreign-key constraints:
    "item_r_id_fkey" FOREIGN KEY (r_id) REFERENCES restaurent(r_id)
Referenced by:
    TABLE "menu" CONSTRAINT "menu_i_id_fkey" FOREIGN KEY (i_id) REFERENCES item(i_id)
```

4.3 Menu

```
postgres=# \d menu;
```

Table "public.menu"				
Column	Type	Collation	Nullable	Default
m_id	character varying(5)		not null	
r_id	character varying(5)			
i_id	character varying(5)			
m_status	character varying(10)			

Indexes:

"menu_pkey" PRIMARY KEY, btree (m_id)

Foreign-key constraints:

"menu_i_id_fkey" FOREIGN KEY (i_id) REFERENCES item(i_id)

"menu_r_id_fkey" FOREIGN KEY (r_id) REFERENCES restaurant(r_id)

Referenced by:

TABLE "orders" CONSTRAINT "orders_m_id_fkey" FOREIGN KEY (m_id) REFERENCES menu(m_id)

TABLE "ordersss" CONSTRAINT "ordersss_m_id_fkey" FOREIGN KEY (m_id) REFERENCES menu(m_id)

4.4 Restaurent

```
postgres=# \d restaurent;
```

Table "public.restaurent"				
Column	Type	Collation	Nullable	Default
r_id	character varying(5)		not null	
r_name	character varying(20)		not null	
cont_no	numeric(10,0)		not null	
r_add	character varying(20)			

Indexes:

"restaurent_pkey" PRIMARY KEY, btree (r_id)

Referenced by:

TABLE "item" CONSTRAINT "item_r_id_fkey" FOREIGN KEY (r_id) REFERENCES restaurent(r_id)

TABLE "menu" CONSTRAINT "menu_r_id_fkey" FOREIGN KEY (r_id) REFERENCES restaurent(r_id)

4.5 Orders

```
postgres=# \d orderssss;
```

Table "public.orderssss"				
Column	Type	Collation	Nullable	Default
o_id	character varying(5)		not null	
c_id	character varying(5)			
m_id	character varying(5)			
o_date	date		not null	

Indexes:

"orderssss_pkey" PRIMARY KEY, btree (o_id)

Foreign-key constraints:

"orderssss_c_id_fkey" FOREIGN KEY (c_id) REFERENCES customer(c_id)

"orderssss_m_id_fkey" FOREIGN KEY (m_id) REFERENCES menu(m_id)

4.6 Delivery_person

```
postgres=# \d delivery_person;
               Table "public.delivery_person"
   Column   |      Type      | Collation | Nullable | Default
-----+-----+-----+-----+-----
 d_id       | character varying(5) |           | not null |
 d_name     | character varying(10) |           |          |
 d_contact  | numeric(10,0)         |           | not null |
 d_add      | character varying(20) |           | not null |
 o_id       | character varying(5) |           |          |
Indexes:
    "delivery_person_pkey" PRIMARY KEY, btree (d_id)
Foreign-key constraints:
    "delivery_person_o_id_fkey" FOREIGN KEY (o_id) REFERENCES orderssss(o_id)
```

4.7 Payment

```
postgres=# \d payment;
               Table "public.payment"
   Column   |      Type      | Collation | Nullable | Default
-----+-----+-----+-----+-----
 p_id       | character varying(5) |           | not null |
 p_date     | date              |           | not null |
 p_status   | character varying(10) |           |          |
 o_id       | character varying(5) |           |          |
 p_mode     | character varying(10) |           |          |
Indexes:
    "payment_pkey" PRIMARY KEY, btree (p_id)
Foreign-key constraints:
    "payment_o_id_fkey" FOREIGN KEY (o_id) REFERENCES orderssss(o_id)
```

5. DATA IMPLEMENTATION

A) SCHEMA

5.1.1 Customer

```
CREATE TABLE customer(  
  c_id VARCHAR(5) PRIMARY KEY,  
  c_name VARCHAR(20) NOT NULL,  
  M_no numeric(10) NOT NULL,  
  email VARCHAR(20),  
  c_add VARCHAR(20) );
```

5.1.2 Item

```
CREATE TABLE item(  
  i_id VARCHAR(5) PRIMARY KEY,  
  i_name VARCHAR(20) NOT NULL,  
  i_price NUMERIC(5),  
  i_category VARCHAR(5),  
  r_id VARCHAR(5) REFERENCES restaurent(r_id));
```

5.1.3 Menu

```
CREATE TABLE menu(  
  m_id VARCHAR(5) PRIMARY KEY,  
  r_id VARCHAR(5) REFERENCES restaurent(r_id),  
  i_id VARCHAR(5) REFERENCES item(i_id),  
  m_status VARCHAR(10) );
```

5.1.4 Restaurent

```
CREATE TABLE restaurent(  
  r_id VARCHAR(5) PRIMARY KEY,  
  r_name VARCHAR(20) NOT NULL,  
  cont_no numeric(10) NOT NULL, r_add VARCHAR(20));
```

5.1.5 orderssss

```
CREATE TABLE orderssss(  
o_id VARCHAR(5) PRIMARY KEY,  
c_id VARCHAR(5) REFERENCES customer(c_id),  
m_id VARCHAR(5) REFERENCES menu(m_id),  
o_date DATE NOT NULL);
```

5.1.6 Delivery_person

```
CREATE TABLE delivery_person(  
d_id VARCHAR(5) PRIMARY KEY,  
d_name VARCHAR(10),  
d_contact NUMERIC(10) NOT NULL,  
d_add VARCHAR(20) NOT NULL,  
o_id VARCHAR(5) REFERENCES orderssss(o_id));
```

5.1.7 Payment

```
CREATE TABLE payment(  
p_id VARCHAR(5) PRIMARY KEY,  
p_date DATE NOT NULL,  
p_status VARCHAR(10),  
o_id VARCHAR(5) REFERENCES orderssss(o_id),  
p_mode VARCHAR(10));
```

B) DATA INSERTION

5.2.1 Customer

```
INSERT INTO customer(c_id,c_name,m_no,email,c_add) VALUES
('C501','FENIL',9373567890,'fenil12@gmail.com','NADIAD'),
('C502','HARSH',9664871124,'harsh21@gmail.com','SURAT'),
('C503','DEEP',8769765346,'deep12@gmail.com','ANAND'),
('C504','VANSI',9824567221,'vansi341@gmail.com','MEHESANA'),
('C505','YASHIL',8769765346,'yashil43@gmail.com','ANAND'),
('C506','RAJ',9313349934,'raj12@gmail.com','NADIAD'),
('C507','NIRAV',8756342134,'nirav12@gamil.com','NADIAD');
```

5.2.2 Item

```
INSERT INTO item(i_id,i_name,i_price,i_category,r_id) VALUES
('I101','PIZZA',250,'VEG','R101'),
('I102','BURGER',120,'VEG','R101'),
('I103','PASTA',180,'VEG','R102'),
('I104','NOODLES',150,'VEG','R103'),
('I105','FRIEDRICE',160,'VEG','R104'),
('I106','SANDWICH',90,'VEG','R105'),
('I107','COLDCOFFEE',80,'DRNK','R106'),
('I108','PANEERTIK',220,'VEG','R107');
```

5.2.3 Menu

```
INSERT INTO menu(m_id,r_id,i_id,m_status) VALUES
('M101','R101','I101','AVAILABLE'),
('M102','R101','I102','AVAILABLE'),
('M103','R102','I103','AVAILABLE'),
('M104','R103','I104','AVAILABLE'),
('M105','R104','I105','AVAILABLE'),
('M106','R105','I106','AVAILABLE'),
('M107','R106','I107','NOTAVAIL'),
('M108','R107','I108','AVAILABLE');
```

5.2.4 Restaurent

```
INSERT INTO restaurent(r_id,r_name,cont_no,r_add) VALUES
('R101','SPICE VILLA','9876543210','NADIAD'),
('R102','FOOD ZONE','9876543211','ANAND'),
('R103','TASTE POINT','9876543212','ANAND'),
('R104','URBAN KITCHEN','9876543213','SURAT'),
('R105','ROYAL DHABA','9876543214','NADIAD'),
('R106','GREEN LEAF','9876543215','SURAT'),
('R107','FOOD COURT','9876543216','MEHESANA'),
('R108','HOTEL DELIGHT','9876543217','MEHESANA'),
('R109','SPICY HUB','9876543218','NADIAD'),
('R110','CITY FOOD','9876543219','ANAND');
```

5.2.5 Orderssss

```
INSERT INTO orderssss(o_id,c_id,m_id,o_date) VALUES
('O101','C501','M101','2026-03-01'),
('O102','C502','M102','2026-03-02'),
('O103','C503','M103','2026-03-03'),
('O104','C504','M104','2026-03-04'),
('O105','C505','M105','2026-03-05'),
('O106','C506','M106','2026-03-06');
```

5.2.6 Delivery_person

```
INSERT INTO delivery_person VALUES
('D101','RAHUL','9876543210','NADIAD','O101'),
('D102','AMIT','9876543211','ANAND','O102'),
('D103','KARAN','9876543212','SURAT','O103'),
('D104','ROHIT','9876543213','MEHESANA','O104');
```

5.2.7 Payment

```
INSERT INTO payment(p_id,p_date,p_status,o_id,p_mode) VALUES
('P101','2026-03-01','PAID','O101','UPI'),
('P102','2026-03-02','PAID','O102','CASH'),
('P103','2026-03-03','PENDING','O103','CARD'),
('P104','2026-03-04','PAID','O104','UPI'),
('P105','2026-03-05','PAID','O105','CASH');
```

INSERTION OUTPUT:

5.2.1 Customer

```
postgres=# Select * From customer;
```

c_id	c_name	m_no	email	c_add
C501	FENIL	9373567890	fenil12@gmail.com	NADIAD
C502	HARSH	9664871124	harsh21@gmail.com	SURAT
C503	DEEP	8769765346	deep12@gmail.com	ANAND
C504	VANSH	9824567221	vansh341@gmail.com	MEHESANA
C505	YASHIL	8769765346	yashil43@gmail.com	ANAND
C506	RAJ	9313349934	raj12@gmail.com	NADIAD
C507	NIRAV	8756342134	nirav12@gamil.com	NADIAD

(7 rows)

5.2.2 Item

```
postgres=# Select * From item;
```

i_id	i_name	i_price	i_category	r_id
I101	PIZZA	250	VEG	R101
I102	BURGER	120	VEG	R101
I103	PASTA	180	VEG	R102
I104	NOODLES	150	VEG	R103
I105	FRIEDRICE	160	VEG	R104
I106	SANDWICH	90	VEG	R105
I107	COLDCOFFEE	80	DRNK	R106
I108	PANEERTIK	220	VEG	R107

(8 rows)

5.2.3 Menu

```
postgres=# SELECT * FROM menu;
 m_id | r_id | i_id | m_status
-----+-----+-----+-----
 M101 | R101 | I101 | AVAILABLE
 M102 | R101 | I102 | AVAILABLE
 M103 | R102 | I103 | AVAILABLE
 M104 | R103 | I104 | AVAILABLE
 M105 | R104 | I105 | AVAILABLE
 M106 | R105 | I106 | AVAILABLE
 M107 | R106 | I107 | NOTAVAIL
 M108 | R107 | I108 | AVAILABLE
(8 rows)
```

5.2.4 Restaurent

```
postgres=# Select * From restaurent;
 r_id | r_name | cont_no | r_add
-----+-----+-----+-----
 R101 | SPICE VILLA | 9876543210 | NADIAD
 R102 | FOOD ZONE | 9876543211 | ANAND
 R103 | TASTE POINT | 9876543212 | ANAND
 R104 | URBAN KITCHEN | 9876543213 | SURAT
 R105 | ROYAL DHABA | 9876543214 | NADIAD
 R106 | GREEN LEAF | 9876543215 | SURAT
 R107 | FOOD COURT | 9876543216 | MEHESANA
 R108 | HOTEL DELIGHT | 9876543217 | MEHESANA
 R109 | SPICY HUB | 9876543218 | NADIAD
 R110 | CITY FOOD | 9876543219 | ANAND
(10 rows)
```

5.2.5 Ordersss

```
postgres=# select * from ordersss;
 o_id | c_id | m_id |   o_date
-----+-----+-----+-----
 0101 | C501 | M101 | 2026-03-01
 0102 | C502 | M102 | 2026-03-02
 0103 | C503 | M103 | 2026-03-03
 0104 | C504 | M104 | 2026-03-04
 0105 | C505 | M105 | 2026-03-05
 0106 | C506 | M106 | 2026-03-06
(6 rows)
```

5.2.6 Delivery_person

```
postgres=# select * from delivery_person;
 d_id | d_name | d_contact | d_add | o_id
-----+-----+-----+-----+-----
 D101 | RAHUL  | 9876543210 | NADIAD | 0101
 D102 | AMIT   | 9876543211 | ANAND  | 0102
 D103 | KARAN  | 9876543212 | SURAT  | 0103
 D104 | ROHIT  | 9876543213 | MEHESANA | 0104
(4 rows)
```

5.2.7 Payment

```
postgres=# SELECT * FROM payment;
 p_id |   p_date   | p_status | o_id | p_mode
-----+-----+-----+-----+-----
 P101 | 2026-03-01 | PAID     | 0101 | UPI
 P102 | 2026-03-02 | PAID     | 0102 | CASH
 P103 | 2026-03-03 | PENDING  | 0103 | CARD
 P104 | 2026-03-04 | PAID     | 0104 | UPI
 P105 | 2026-03-05 | PAID     | 0105 | CASH
(5 rows)
```

5.3 QUERIES USING BASIC DBMS CONSTRUCTS JOIN & SUBQUERIES:

5.3.1 Display employee ID

```
postgres=# SELECT E_ID FROM EMPLOYEE;  
e_id  
-----  
501  
502  
503  
504  
505  
506  
507  
508  
509  
510  
(10 rows)
```

5.3.2 Display employee name who lives in nadiad.

```
postgres=# SELECT E_name FROM employee WHERE E_address='NADIAD';  
e_name  
-----  
AJAY  
KIYARA  
MIHIR  
(3 rows)
```

5.3.3 Display blood stock before date 2022-09-20.

```
postgres=# SELECT * FROM blood_stock WHERE BS_date<'2022-09-20';
```

bs_id	bs_date	i_id
1901	2022-09-01	
1902	2022-09-02	
1903	2022-09-03	
1904	2022-09-04	
1905	2022-09-05	
1906	2022-09-06	
1907	2022-09-07	
1908	2022-09-08	
1909	2022-09-09	
1910	2022-09-10	

(10 rows)

5.3.4 Display employee details with salary greater than 50000.

```
postgres=# SELECT * FROM employee e inner join salary s on e.S_id=s.S_id where s.Salary>50000;
```

e_id	a_id	e_name	e_address	e_phoneno	e_dob	s_id	s_id	salary
501	101	AJAY	NADIAD	9898160331	2012-09-04	2102	2102	60000
503	102	SHIVA	SURAT	7898763523	2013-07-09	2110	2110	100000
504	103	STACY	BARODA	7987653432	2013-06-12	2108	2108	88000
506	102	DHYEY	AHEMDABAD	6733737373	2015-06-02	2105	2105	90000
507	103	MIHIR	NADIAD	6574323342	2015-06-09	2104	2104	66000
508	101	HIL	SURAT	7865653432	2013-09-03	2107	2107	100000
510	102	OM	SURAT	8769765346	2015-09-08	2109	2109	85000

(7 rows)

5.3.5 Display blood details where blood group is AB+ve.

```
postgres=# SELECT * FROM blood where B_Group='AB+';
```

b_id	b_group
203	AB+
212	AB+
216	AB+

(3 rows)

5.3.6 Display count of blood having blood group AB+ve.

```

postgres=# SELECT count(B_Group) FROM blood where B_Group='AB+';
count
-----
      3
(1 row)

```

5.3.7 Display employee name which employee number is highest.

```

postgres=# select *from employee WHERE E_id=(select max(E_id) FROM employee);
 e_id | a_id | e_name | e_address | e_phoneno |  e_dob   | s_id
-----+-----+-----+-----+-----+-----+-----
  510 |  102 | OM     | SURAT     | 8769765346 | 2015-09-08 | 2109
(1 row)

```

5.3.8 Display avg salaey of employee.

```

postgres=# SELECT avg(salary) from salary;
avg
-----
70700.0000000000000000
(1 row)

```

5.4.9 Display the donor count grouped by blood group.

```
postgres=# select count(b.B_Group),b.B_Group as doner_group from blood b inner join doner d on b.B_id=d.B_id group by b.B_Group;
count | doner_group
-----+-----
1      | AB-
1      | AB+
2      | A+
1      | B-
2      | B+
1      | A-
2      | O+
(7 rows)
```

5.4.10 Display name of employee and the query associated with him/her.

```
postgres=# select e.E_name,h.HD_Query from hd_emp he inner join help_desk h on he.HD_id= h.HD_id inner join employee e on he.E_id=e.E_id;
e_name | hd_query
-----+-----
HIL    | BLOOD BOTTLE LOST
MIHIR  | PAYMENT FAILED
MIHIR  | IMPROPER SERVICE
SHIVA  | WRONG BLOOD GROUP
SUMAN  | DONOR NOT FOUND
STACY  | BLOOD GROUP MISMATCH
HIL    | PAYMENT FAILED
(7 rows)
```

5.4.11 Display id and name of employee in asc order whose salary is greater 50000 and id lies between 501 to 507

```

postgres=# SELECT E_id,E_name
postgres=# from
postgres=# employee
postgres=# where (E_id between 501 and 507
postgres=# AND S_id in (
postgres=#     select
postgres=#     S_id
postgres=#     from
postgres=#     salary
postgres=#     where
postgres=#     salary>50000
postgres=# ))
postgres=# order by
postgres=# E_name ASC;
 e_id | e_name
-----+-----
  501 | AJAY
  506 | DHYEV
  507 | MIHIR
  503 | SHIVA
  504 | STACY
(5 rows)

```

5.5 PL/SQL(VIEWS):

Create view with employee name and address and display all details from it.

```

postgres=# create view E_name_address as select E_id,E_address from employee;
CREATE VIEW
postgres=# select * from E_name_address;
 e_id | e_address
-----+-----
  501 | NADIAD
  502 | NADIAD
  503 | SURAT
  504 | BARODA
  505 | ANAND
  506 | AHMEDABAD
  507 | NADIAD
  508 | SURAT
  509 | BARODA
  510 | SURAT
(10 rows)

```

5.6 FUNCTION & TRIGGERS:

5.6.1 Create a trigger to get new salary input which should not be less than 50000

Function:

```
create function check_sal() returns trigger as $$
BEGIN
  if NEW.salary<50000 then
    raise exception 'Salary Must be greater than 50000'; end
  if;
  return NEW;
  END;
$$
LANGUAGE plpgsql;
```

Trigger:

```
CREATE trigger sal_check
BEFORE INSERT OR UPDATE
ON salary
FOR EACH ROW
```

EXECUTE PROCEDURE check_sal();

```
postgres=# create function check_sal() returns trigger as $$
postgres$$ BEGIN
postgres$$ if NEW.salary<50000 then
postgres$$ raise exception 'Salary Must be greater than 50000';
postgres$$ end if;
postgres$$ return NEW;
postgres$$ END;
postgres$$ $$
postgres-# LANGUAGE plpgsql;
CREATE FUNCTION
postgres=# create trigger sal_check
postgres-# BEFORE INSERT OR UPDATE
postgres-# ON salary
postgres-# FOR EACH ROW
postgres-# EXECUTE PROCEDURE check_sal()
postgres-# ;
CREATE TRIGGER
```

Check: insert into salary values (2111,19000);

```
postgres=# insert into salary values (2111,19000);
ERROR: Salary Must be greater than 50000
CONTEXT: PL/pgSQL function check_sal() line 4 at RAISE
```

5.6.2 Create a function to count numbers of employees with salary between a user defined range.

Function:

```
CREATE function employeewithsalarybetween(salaryfrom int , salaryto int)
returns int
language plpgsql
as $$
declare numberofemp
integer;
begin select
count(*) into
numberofemp
from salary
where salary between salaryfrom and salaryto;
return numberofemp;
end; $$;
```

Check: select employeewithsalarybetween(50000,90000);

```
postgres=# CREATE function employeewithsalarybetween(salaryfrom int , salaryto int)
postgres=# returns int
postgres=# language plpgsql
postgres=# as
postgres=# $$
postgres$# declare
postgres$# numberofemp integer;
postgres$# begin
postgres$# select count(*)
postgres$# into numberofemp
postgres$# from salary
postgres$# where salary between salaryfrom and salaryto;
postgres$# return numberofemp;
postgres$# end;
postgres$# $$;
CREATE FUNCTION
postgres=# select employeewithsalarybetween(50000,90000);
employeewithsalarybetween
-----
6
(1 row)
```

5.6.3 Create a trigger for insertion of new employee details which should not belong to nadiad

Function :

```
CREATE function check_emp_city4() returns trigger as $$
BEGIN
  if(NEW.E_address='Nadiad') then
    raise exception 'Employee must not belongs to nadiad!'; end
  if;
return NEW;
END;
$$
LANGUAGE plpgsql;
```

Trigger:

```
create trigger city_check4
BEFORE INSERT OR UPDATE
ON employeeed
FOR EACH ROW
EXECUTE PROCEDURE check_emp_city4();
```

Check: insert into employeeed(E_id,A_id,E_address,E_phoneno,E_dob,s_id)
values(522,101,'AJAY','nadiad',989816031,'2012-09-04',2102);

```
postgres=# create function check_emp_city4() returns trigger as $$
postgres## BEGIN
postgres##   if(NEW.E_address='Nadiad') then
postgres##     raise exception 'Employee must not belongs to nadiad!';
postgres##   end if;
postgres## return NEW;
postgres## END;
postgres## $$
postgres-# LANGUAGE plpgsql;
CREATE FUNCTION
postgres=#
postgres=# create trigger city_check4
postgres-# BEFORE INSERT OR UPDATE
postgres-# ON employeeed
postgres-# FOR EACH ROW
postgres-# EXECUTE PROCEDURE check_emp_city4();
CREATE TRIGGER
postgres=# INSERT INTO employeeed (E_id, A_id, E_name, E_address, E_phoneno, E_dob, S_id) VALUES
postgres-# (522, 101, 'AJAY', 'Nadiad', 989816031, '2012-09-04', 2102);
ERROR:  Employee must not belongs to nadiad!
CONTEXT:  PL/pgSQL function check_emp_city4() line 4 at RAISE
```

5.7 CURSOR:

Create a cursor to select employee ID with greater than 501 and fetch the next employee ID

CURSOR SYNTAX:

Begin;

declare my_e_id cursor for

select e_id from employee where e_id > 501;

fetch next from my_e_id; close my_e_id;

cursor ;

```
postgres=# begin;
BEGIN
postgres=# declare my_e_id cursor for
postgres=# select e_id from employee where e_id > 501;
DECLARE CURSOR
postgres=# fetch next from my_e_id;
 e_id
-----
   502
(1 row)

postgres=# close my_e_id;
CLOSE CURSOR
```